

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД  
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»  
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ  
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ  
до лабораторної роботи №5  
з дисципліни «Програмування систем на кристалі»

Студента 4-го курсу  
спеціальності 123 –  
«Комп'ютерна інженерія»  
Голубки Миколи

м. Ужгород – 2026 р.

## Лабораторна робота №5

**Тема:** Динамічна індикація, таймери, робота з логічним аналізатором у інтегрованому середовищі PSoC® Creator™

**Мета:** Ознайомитися з принципами роботи динамічної індикації та налаштуванні переривань у мікроконтролерних системах на базі PSoC. Набути практичних навичок створення проєктів у інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів для керування індикаторами через зсувні регістри та реалізації ефективного виведення даних. Також набути практичних навичок використання логічного аналізатора на прикладі Saleae Logic.

### Завдання

Написати код таким чином, щоб семисегментний дисплей виводилось правильно чи неправильно введений пароль, введення паролю з матричної клавіатури і діод щоб загорався зеленим коли правильно введений пароль, а червоний — неправильно введений пароль.

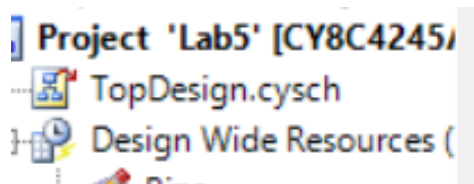


Рис. 1.1

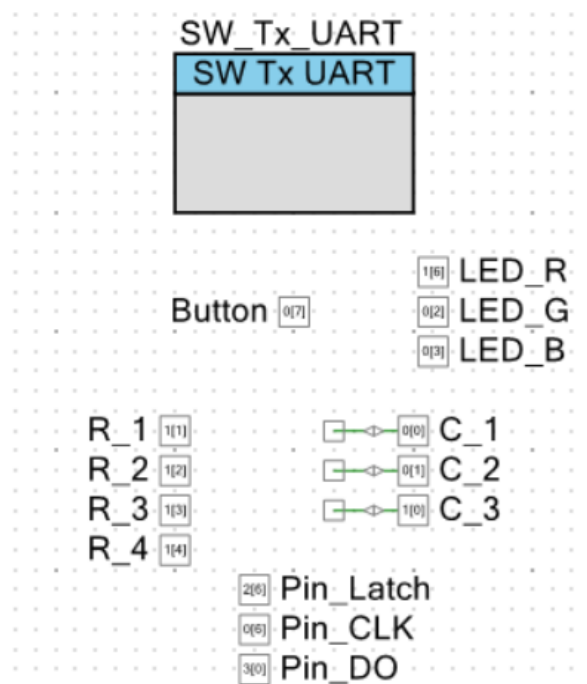


Рис. 1.2

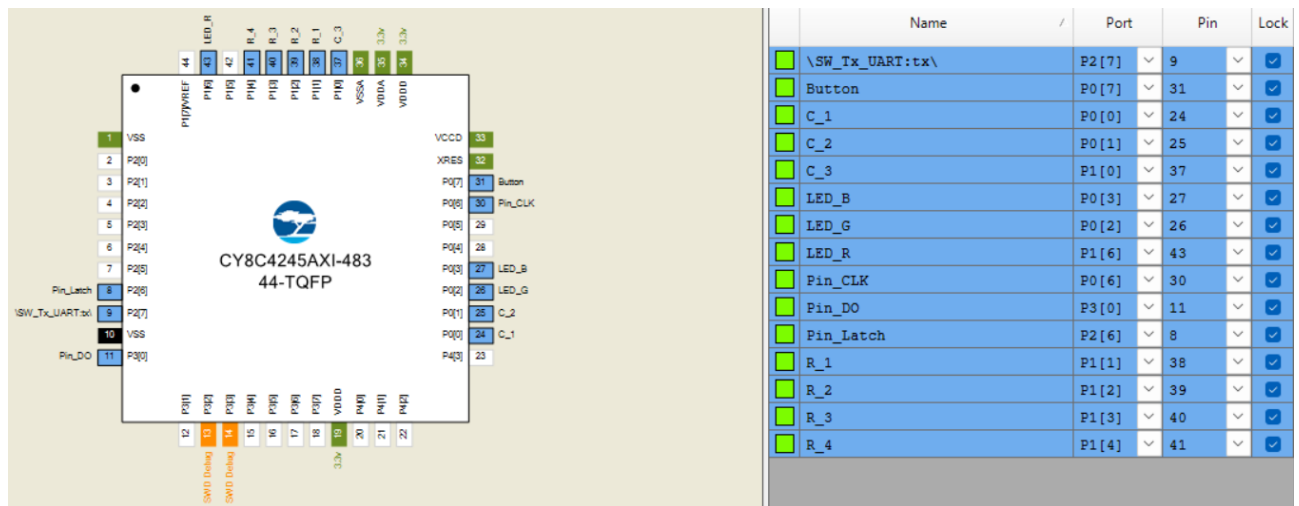


Рис. 1.3

### 3. Написать код для вывода данных.

```
#include "project.h"
#include <string.h>

#define PASSWORD_LENGTH 4
const char correct_password[PASSWORD_LENGTH + 1] = "5578";
char input_buffer[PASSWORD_LENGTH + 1] = {0};
uint8_t input_idx = 0;

#define SEG_BLANK 0xFF
#define SEG_MINUS 0xBF
#define SEG_P      0x8C
#define SEG_A      0x88
#define SEG_S      0x92
#define SEG_F      0x8E
#define SEG_I      0xF9
#define SEG_L      0xC7

const uint8_t DIGIT_MAP[12] = {
    0xC0, 0xF9, 0xA4,
    0xB0, 0x99, 0x92,
    0x82, 0xF8, 0x80,
    0x88, 0x90, 0x83
};

};

uint8_t display_buf[8] = {SEG_BLANK, SEG_BLANK, SEG_BLANK, SEG_BLANK,
    SEG_BLANK, SEG_BLANK, SEG_BLANK, SEG_BLANK};

void SetLED(uint8_t r, uint8_t g, uint8_t b) {
    LED_R_Write(r);
    LED_G_Write(g);
    LED_B_Write(b);
}

void SendTo595(uint8_t data) {
    for(int8_t i = 0; i < 8; i++) {
        if (data & (0x80 >> i)) {
            Pin_DO_Write(1);
        } else {
            Pin_DO_Write(0);
        }
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

void DisplayRefresh(void) {
```

```

    for (uint8_t i = 0; i < 8; i++) {
        SendTo595(0xFF & ~(1 << i));
        SendTo595(display_buf[i]);

        Pin_Latch_Write(1);
        Pin_Latch_Write(0);

        CyDelay(2);
    }
}

void DelayWithDisplay(uint32_t ms) {

    uint32_t loops = ms / 16;
    for(uint32_t i = 0; i < loops; i++) {
        DisplayRefresh();
    }
}

char ScanKeypad(void) {
    char key = 0;

    C_1_Write(0); C_2_Write(1); C_3_Write(1); CyDelayUs(10);
    if (R_1_Read() == 0) key = '1';
    if (R_2_Read() == 0) key = '4';
    if (R_3_Read() == 0) key = '7';
    if (R_4_Read() == 0) key = '*';

    C_1_Write(1); C_2_Write(0); C_3_Write(1); CyDelayUs(10);
    if (R_1_Read() == 0) key = '2';
    if (R_2_Read() == 0) key = '5';
    if (R_3_Read() == 0) key = '8';
    if (R_4_Read() == 0) key = '0';

    C_1_Write(1); C_2_Write(1); C_3_Write(0); CyDelayUs(10);
    if (R_1_Read() == 0) key = '3';
    if (R_2_Read() == 0) key = '6';
    if (R_3_Read() == 0) key = '9';
    if (R_4_Read() == 0) key = '#';

    C_1_Write(1); C_2_Write(1); C_3_Write(1);

    return key;
}

void ClearSystem(void) {
    input_idx = 0;
    for(int i=0; i<8; i++) display_buf[i] = SEG_BLANK;
    SetLED(0, 0, 0);
}

int main(void) {
    CyGlobalIntEnable;
    SW_Tx_UART_Start();
    SW_Tx_UART_PutString("\r\n Lab5 Security System Started \r\n");
    char key = 0;
    char last_key = 0;

    ClearSystem();
    for(;;) {
        key = ScanKeypad();
        if (key != 0 && last_key == 0) {

            if (key == '#') {

                if (input_idx == PASSWORD_LENGTH && strcmp(input_buffer,
correct_password, PASSWORD_LENGTH) == 0) {
                    SW_Tx_UART_PutString("\r\nACCESS GRANTED\r\n");

```

```

        SetLED(1, 0, 1);

        display_buf[0] = SEG_MINUS;
        display_buf[1] = SEG_BLANK;
        display_buf[2] = SEG_P;
        display_buf[3] = SEG_A;
        display_buf[4] = SEG_S;
        display_buf[5] = SEG_S;
        display_buf[6] = SEG_BLANK;
        display_buf[7] = SEG_MINUS;

        DelayWithDisplay(2500);
        ClearSystem();

    } else {

        SW_Tx_UART_PutString("\r\nACCESS DENIED\r\n");
        SetLED(0, 1, 1);

        display_buf[0] = SEG_BLANK;
        display_buf[1] = SEG_MINUS;
        display_buf[2] = SEG_F;
        display_buf[3] = SEG_A;
        display_buf[4] = SEG_I;
        display_buf[5] = SEG_L;
        display_buf[6] = SEG_MINUS;
        display_buf[7] = SEG_BLANK;

        DelayWithDisplay(2500);
        ClearSystem();
    }

} else if (key == '*') {
    ClearSystem();
} else {
    if (input_idx < PASSWORD_LENGTH) {
        input_buffer[input_idx] = key;
        display_buf[input_idx] = DIGIT_MAP[key - '0'];
        input_idx++;
        SW_Tx_UART_PutChar(key);
    }
}

last_key = key;

DisplayRefresh();
}
}

```

4. Скомпілювати та запрограмувати проєкт в PSoC® 4 Pioneer Kit.

Build → Compile File (Ctrl+F6) Build → Build (Shift+F6) Debug → Program (Ctrl+F5)

6. Збережіть проєкт.